

Large Language Model as Token Compressor and Decompressor

Wenbing Li Yiran Wang Zikai Song Jielei Zhang Tianhao Zhao
 Junkai Lin Huipeng Guo Wei Yang*

Abstract

In this paper, we study whether an off-the-shelf LLM can be adapted into a discrete, variable-length token compressor and decompressor for long-context processing. To this end, we design a self-expressive autoencoding framework that fine-tunes a pretrained LLM with lightweight LoRA adapters to map long texts into compact sequences of learned latent codes, termed Z-tokens, and to decode them back into natural language or task outputs. The resulting representation is content-adaptive: less predictable or information-dense segments can receive more Z-tokens, while redundant regions can be represented more compactly through a budget-aware length regularizer. Our method is evaluated on long-context datasets such as Wikipedia, CNN/DailyMail, HotpotQA, and QuALITY, showing that it preserves reconstruction quality and downstream performance while reducing effective context length, generation-stage memory usage, and end-to-end latency. This simple design supports both direct decoding from compressed contexts and autoregressive generation in the Z-token space, providing a practical interface for efficient long-context inference.

1 Introduction

Large language models (LLMs) have achieved remarkable success in various fields [1, 2]. However, their ability to handle long contexts remains fundamentally limited by the quadratic complexity of the attention mechanism [6]. As sequence lengths grow to tens or even hundreds of thousands of tokens, the computational and memory overhead of the attention mechanism makes large-scale inference, long document understanding, and multi-hop retrieval prohibitively expensive. To mitigate this, significant research has focused on token compression, which aims to distill lengthy sequences into shorter, information-dense representations [7, 8, 9, 10]. Existing approaches can be broadly categorized into two categories. Heuristic token merging methods, such as similarity-based merging [13, 14], attention-based pruning [15], and saliency-guided aggregation [16], attempt to remove or merge redundant tokens based on local similarity or attention importance scores. While these techniques are effective in shortening sequence lengths, tokens that appear redundant due to similarity may carry discourse or pragmatic meaning that is difficult to recover. The second category includes learning-based or autoencoding methods such as AutoCompressor [9], ICAE [7], or Gist Tokens [10]. AutoCompressor and ICAE use LLMs to generate fixed-length compressed context vectors that can be re-expanded for downstream tasks by training auxiliary networks or soft embedding modules. Gist Tokens force the model to compress cues into fixed-length lexical units through an attention masking mechanism. These methods often treat compression as an external, auxiliary task, decoupled from the LLM’s primary generative reasoning process. Moreover, their reliance on fixed compression ratios prevents adaptation to varying information density and typically forgoes faithful reconstruction of the original input, limiting their applicability in scenarios that demand verifiability or fine-grained access to the source content. In this work, we depart from prevailing heuristic and learned-embedding approaches by introducing a novel insight: leveraging the semantic knowledge of the LLM to serve as both a token compressor and decompressor. Instead of representing the compressed context as

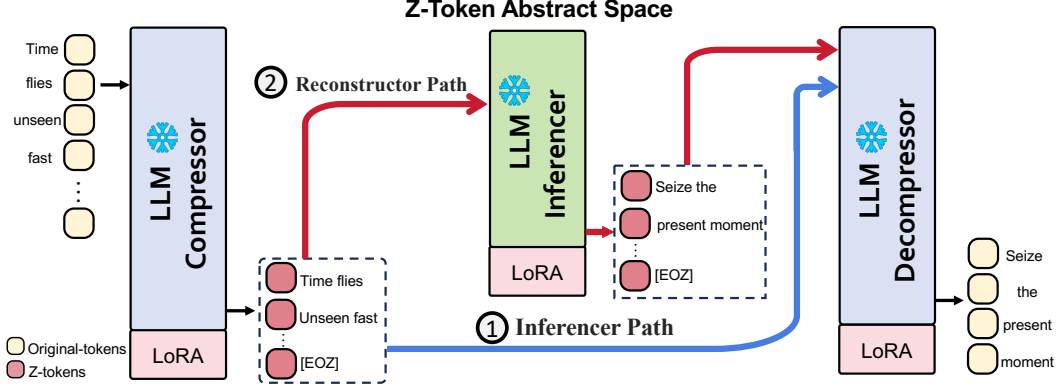


Figure 1: Our framework consists of three components: a compressor, an inference module, and a decompressor. It supports two usage paradigms. In the first, we directly feed the compressed representation to the decompressor to perform the downstream task. In the second, we first run the inference module in the Z-token space and then invoke the decompressor to recover task outputs at the token level.

a continuous vector or a fixed number of memory slots, we expand the LLM vocabulary using the learned Z-token codebook and generate compressed tokens autoregressively. Our method is predicated on the fundamental insight that LLMs, trained on vast corpora, are not merely statistical models but powerful engines for semantic abstraction and conceptual distillation [17, 18, 19]. We observe that natural language is inherently compressive; it exhibits a strong tendency to encode complex ideas into compact lexical units. Idioms such as “double-edged sword” encapsulate intricate trade-offs, and metaphors like “tip of the iceberg” summarize extensive underlying complexities. This phenomenon implies that LLMs themselves are capable of efficient compression. Methodologically, we propose an autoencoding training paradigm, where an input sequence X is mapped into a compressed sequence of discrete Z-tokens, Z , by a LLM compressor, and subsequently reconstructed or continued by a LLM decompressor. The training objective can be viewed as a form of bilingual translation, where the model learns to encode natural language into its own abstract internal language and decode it back. We make this process explicit with autoregressive discrete generation over an extended codebook, plus a budget-aware length regularizer that controls the realized compression ratio. The compressed latent $\{Z\}$ serves as a semantic scaffold, forcing the model to perform core reasoning on compact symbolic representations before projecting them back to surface tokens. Our design is more flexible, capable of dynamically lengthening compressed text based on semantic density, and seamlessly integrating with sliding windows to support longer contexts. We further analyze the contextual regularity of the learned Z-token codebook and show that repeated Z-tokens tend to occur in semantically related contexts, while still remaining context-dependent rather than fixed lexical units. We conducted extensive experiments on six large-scale datasets—Wikipedia, CNN/Daily Mail, HotpotQA, NarrativeQA, QASPER, and QuALITY—covering text reconstruction, continuation, summarization, and long text question answering, demonstrating the effectiveness of our method.

2 Related Works

Context Compression refers to any mechanism that explicitly aims to reduce the number of tokens participating in subsequent computation. Some approaches follow *heuristic token reduction* strategies, removing or merging less informative tokens based on saliency or similarity. Token Merging [13, 24, 14] aggregates similar tokens, while Token Pruning [15, 14, 29, 30, 31] eliminates low-saliency tokens during inference. Beyond rule-based methods, *learning-based semantic compression* encodes contexts into compact representations that retain high-level meaning. ICAE [7] and AutoCompress [9] represent long sequences as latent vectors for efficient reconstruction or reasoning; other recent works similarly investigate learned context compression [8, 35]. A related branch, *prompt compression*, targets input instructions rather than the full context. Methods such as 500xCompressor [40], Gist Tokens [10] and LongLLMLingua [41] summarize long prompts into shorter, reusable representations. Our method is conceptually aligned with *learning-based semantic compression* but differs in execution: rather than encoding context externally, we integrate compression into the LLM’s reasoning process. This yields discrete internal symbols that form a self-expressive “semantic language,” achieving adaptive, interpretable, and semantically faithful compression.

Architectural and Implicit Sparsity Methods represent an alternative line of research that, instead of explicitly reducing the number of tokens involved in computation, improve efficiency through architectural modifications or implicitly introduced sparsity in the computation process. Compressive Transformer [42] introduces hierarchical memory buffers summarizing distant activations; Reformer [43] leverages locality-sensitive hashing to approximate attention; Routing Transformer [44] sparsifies attention through content-based routing. Other models such as Longformer, BigBird, and Window Attention [45, 46, 47, 49, 50, 51, 53] also achieve scalability via structured sparsity.

Discrete Semantic Representation provides a principled foundation for compact and interpretable encoding of high-level semantics. VQ-VAE [54] introduced vector-quantized bottlenecks that map continuous signals into a learned codebook, later extended to sequence models [55, 56, 57, 58, 59] and cross-modal learning frameworks [60]. These approaches show that discrete latent variables can serve as symbolic carriers of information. Subsequent works explored training stability and optimization for discrete variables [63]. While prior works used discrete bottlenecks as auxiliary modules, we internalize this semantic layer within the LLM’s reasoning flow, enabling discrete tokens to serve as learned, interpretable units for compressing and reasoning over long contexts.

3 Methods

Our goal is to adapt a pretrained LLM into a compressor and decompressor. The compression and decompression work as follows: given an input sequence $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, the compressor generates a variable-length sequence of tokens $\mathbf{Z} = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_K\}$, where $K \ll N$. We consider two usage modes: 1). **Direct decompression for inference**: the decompressor directly consumes Z-tokens and generates downstream task outputs, so the compressor acts as a learned prompt compressor; 2). **Z-space inference followed by reconstruction**: the decompressor reconstructs surface text from Z-tokens, while a separate inferencer operates in the Z-token space before final decoding. Since the compressor maps an N -token input to a much shorter Z-token sequence of length $K \ll N$, subsequent attention-based computation scales as $\mathcal{O}(K^2)$ instead of $\mathcal{O}(N^2)$, yielding a theoretical compute and memory saving of approximately $(N/K)^2$ and enabling faster inference, lower memory usage, and effectively longer contexts.

3.1 LLM Compressor

Given an input token sequence $\mathbf{X}$, the compressor generates a variable-length Z-token sequence $\mathbf{Z}$. To preserve the prior of the pretrained LLM, we extend the original vocabulary to $[\mathcal{V}_{\text{base}}, \mathcal{V}_z]$, where $\mathcal{V}_{\text{base}}$ is the original vocabulary and $\mathcal{V}_z$ is the extended vocabulary for Z-tokens. Z-tokens are generated only from $\mathcal{V}_z$. Unlike previous work (e.g., ICAE or AutoCompressor) that directly learns fixed-length soft embeddings, our compressor generates $\mathbf{Z}$ autoregressively:

$$p_\phi(\mathbf{Z} | \mathbf{X}) = \prod_{t=1}^K p_\phi(z_t | \mathbf{X}, z_1, \dots, z_{t-1}). \quad (1)$$

The compressed length K is not fixed a priori but determined dynamically by the compressor itself. Concretely, we augment $\mathcal{V}_z$ with a special [EOS-Z] token; generation stops once the compressor emits [EOS-Z], so K emerges from the model output rather than being imposed as a hyperparameter. To avoid degenerate solutions in which the model either collapses to a single Z-token or reproduces the input length, we regularize the realized compression ratio against a target ratio r :

$$\mathcal{L}_{\text{len}} = \left(\frac{K}{N} - \frac{1}{r} \right)^2 \quad (2) \quad z_t = \text{GumbelSoftmax}(\mathbf{h}_t), \quad z_t \in \mathcal{V}_z \quad (3)$$

where N is the input length. This soft constraint allows informative segments to consume more Z-tokens while penalizing global budget violations, enabling content-adaptive allocation without imposing rigid per-segment quotas. This design can also be interpreted from a constrained information allocation perspective. Suppose $\mathbf{X}$ is partitioned into local segments $\{\mathbf{X}^{(i)}\}_{i=1}^M$, and the compressor allocates K_i latent tokens to segment $\mathbf{X}^{(i)}$ under a global budget $\sum_i K_i \leq N/r$. If the segment-wise information density is defined as

$$H_i \triangleq H(\mathbf{X}^{(i)} | \mathbf{X}^{(<i)}),$$

and the distortion for segment $\mathbf{X}^{(i)}$ takes the form $D_i(K_i) = H_i e^{-\beta K_i}$ with $\beta > 0$, then the optimal allocation satisfies

$$K_i^* = \frac{1}{\beta} (\log(\beta H_i) - \log \lambda),$$

for some $\lambda > 0$, and is therefore monotonically increasing in H_i . In other words, under a fixed compression budget, semantically denser or less predictable segments should receive more Z-tokens, while more redundant segments should receive fewer. From this perspective, the [EOS-Z] mechanism together with $\mathcal{L}_{\text{len}}$ implements a soft rate-distortion trade-off. A complete derivation and discussion are provided in Appendix D. Therefore, the model can dynamically determine the lexical units needed to represent a given context, achieving variable-length compression in a way that remains compatible with the native generation process of LLMs. To ensure end-to-end training while maintaining discreteness, we use a Gumbel-Softmax estimator to sample Z-tokens only from $\mathcal{V}_z$. This produces a soft embedding e_{soft} during training. We then apply a straight-through estimator to combine differentiable gradients with discrete decoding:

$$e = e_{\text{hard}} + [e_{\text{soft}} - \text{sg}(e_{\text{soft}})] \quad (4) \quad \mathcal{L}_{\text{overlap}} = 1 - \cos(\mathbf{Z}_i^{\text{overlap}}, \mathbf{Z}_{i+1}^{\text{overlap}}) \quad (5)$$

where e_{hard} denotes the embedding obtained by argmax. This hybrid estimator maintains stable gradient flow without sacrificing the discreteness of the learned symbols.

Sliding window compression. For excessively long sequences, our method naturally adapts to sliding-window compression: the input sequence is segmented into overlapping windows of length W and stride S (typically $W = 1024$ and $S = 256$), each independently compressed into $\mathbf{Z}_i$. For adjacent segments, we enforce a consistency constraint on overlapping regions to ensure semantic coherence. We can then concatenate tokens of compressed segments $\{\mathbf{Z}_i\}$ for long-context reconstruction or inference.

3.2 Direct decompression for inference

The LLM decompressor serves as the core component for translating the compressed Z-tokens back to meaningful semantic outputs. Our decompressor supports two distinct usage paradigms: the inferencer and the reconstructor. For both paradigms, the decompression process is modeled as conditional autoregressive generation, with shared training objectives and constraints that ensure stability and semantic fidelity. Formally, the decompression process follows the distribution:

$$p_{\theta}(\mathbf{Y}|\mathbf{Z}) = \prod_{t=1}^T p_{\theta}(y_t | y_1 \dots y_{t-1}, \mathbf{Z}), \quad (6)$$

Depending on the usage paradigm, the sequence $\mathbf{Y}$ varies. For the reconstructor task, $\mathbf{Y}$ is the input sequence to the compressor $\mathbf{X}$. While for the inferencer task, such as Question-Answering (QA), $\mathbf{Y}$ is the response to $\mathbf{X}$. $\mathbf{Z}$ represents the Z-tokens generated by the compressor. To ensure a linguistic foundation, the decompressor is trained under lexical constraints: in each decoding step, the output logit vector is restricted to within the pre-trained model’s original vocabulary $\mathcal{V}_{\text{base}}$:

$$\tilde{p}_{\theta}(y_t | y_1 \dots y_{t-1}, \mathbf{Z}) = \text{Softmax}(W_{\text{base}} \mathbf{h}_t), \quad (7)$$

where $\mathbf{h}_t$ represents the decoder’s hidden state at step t , and W_{base} is the embedding matrix of the original vocabulary set. This constraint forces the model to use human-understandable vocabulary to express internal abstract representations, effectively translating the private latent “language” back into natural language. Empirical evidence shows this vocabulary constraint improves training stability and ensures semantic fidelity across tasks. For training, the general form of the loss is a cross-entropy term that maximizes the likelihood of generating correct sequence given the Z-tokens:

$$\mathcal{L}_{\text{tr}} = - \sum_{t=1}^T \log p_{\theta}(y_t | y_1 \dots y_{t-1}, \mathbf{Z}), \quad (8)$$

Training entirely under teacher forcing introduces exposure bias, where the model learns to rely on ground-truth labels that are unavailable at inference time. To mitigate this mismatch, we adopt a *scheduled sampling* strategy: at each decoding step, the lexical input is taken from the ground-truth token with probability $1 - p$, or from the model’s own previous prediction with probability p . The probability p increases linearly during training, allowing the model to gradually transition from teacher-forced decoding to free decoding. This improves robustness to its own prediction errors and leads to better generalization at inference time.

In addition to the token-level reconstruction objective, we introduce two regularization terms to stabilize discrete code learning and promote effective use of the Z-token vocabulary. The first term, $\mathcal{L}_{\text{com}}$, regularizes the consistency between the soft embedding selected by Gumbel-Softmax and its corresponding hard embedding:

$$\mathcal{L}_{\text{com}} = \mathbb{E}_t \left[\|\text{sg}(\mathbf{e}_t^{\text{soft}}) - \mathbf{e}_t^{\text{hard}}\|_2^2 \right] + \eta \mathbb{E}_t \left[\|\mathbf{e}_t^{\text{soft}} - \text{sg}(\mathbf{e}_t^{\text{hard}})\|_2^2 \right]. \quad (9)$$

Here, $\text{sg}(\cdot)$ denotes the stop-gradient operator. This term stabilizes discrete code learning by penalizing the discrepancy between soft and hard embeddings. The second term, $\mathcal{L}_{\text{KL}}$, prevents codebook collapse by encouraging more balanced usage of the Z-token vocabulary:

$$\mathcal{L}_{\text{KL}} = \sum_{j=1}^{|\mathcal{V}_z|} \hat{\mathbf{q}}_j \left(\log(\hat{\mathbf{q}}_j + \epsilon) - \log(u_j + \epsilon) \right), \quad (10)$$

where $\hat{\mathbf{q}} = \frac{1}{M} \sum_{m=1}^M \tilde{\mathbf{q}}^{(m)}$ is the average soft assignment over a minibatch of size M , and u_j denotes the target uniform prior over $\mathcal{V}_z$. This regularizer encourages the compressor to use the extended codebook more evenly, thereby avoiding degenerate solutions in which only a small subset of Z-tokens is activated. The final training objective for the decompressor integrates the reconstruction loss with these regularization terms to ensure end-to-end consistency:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{tr}} + \lambda \mathcal{L}_{\text{KL}} + \beta \mathcal{L}_{\text{com}} + \gamma \mathcal{L}_{\text{len}}, \quad (11)$$

where λ , β , and γ control the strengths of the code-usage, embedding-consistency, and length regularizers, respectively.

3.3 Z-space inference followed by reconstruction

The decompressor as reconstructor paradigm positions the decompressor as a reconstructor of the input text to the compressor from Z-tokens, while a separate LLM operates in the Z-token space between compression and reconstruction to enable reasoning purely in the Z-Token space. This design decouples compression from high-level reasoning, allowing the inferencer LLM to focus on high-level semantics without handling redundant surface-level tokens. In this case, we first train the decompressor using Eqn. 11 with the target sequence $\mathbf{Y}$ as the compressor input $\mathbf{X}$. Then we compress both the prompt sequence $\mathbf{X}$ and the response sequence $\mathbf{Y}$ into Z-Token sequences $\mathbf{Z}^p$ and $\mathbf{Z}^r$. The separate LLM inferencer conducts autoregressive generation entirely on Z-tokens, treating them as an internal symbolic language, as:

$$p_\psi(\mathbf{Z}^r | \mathbf{Z}^p) = \prod_{t=1}^{|\mathbf{Z}^r|} p_\psi(\mathbf{Z}_t^r | \mathbf{Z}^p, z_1^r \dots z_{t-1}^r), \quad (12)$$

where ψ represents the parameters of the inference LLM trained or adapted for symbolic reasoning. This allows operations such as summarization, continuation, and question answering to be performed in the latent space without returning to surface tokens. A key advantage of reasoning in the Z-Token space lies in its semantic density and computational efficiency. Each Z-Token encodes multiple surface tokens, enabling the model to infer long-range dependencies with significantly reduced context length. Essentially, this paradigm treats Z-tokens as a symbolic interface between compression and reasoning; the model no longer needs to reconstruct before thinking, but can think directly using its own abstract language. This provides a new window into how LLMs internally represent and manipulate semantic knowledge.

3.4 Contextual Regularity of Z-tokens

We observe that the learned codebook $\mathcal{V}_Z$ is reused in heterogeneous inputs: the same Z-tag index can appear in different contexts but yields different representations after decompression; conversely, the same input may have multiple valid Z sequences at different sampling temperatures. Unlike traditional tokenizers, where one token corresponds to one subword, this suggests that Z-tokens function as abstract, context-dependent semantic units, rather than deterministic mappings.

Qualitative illustrations. In several cases, Z-tokens operate at a conceptual level rather than encoding fixed surface wording. 1) **Semantic synonymy:** sentences such as “*The government implemented sweeping economic reforms to address the financial crisis*” and “*Officials enacted major fiscal restructuring to mitigate the debt emergency*” share overlapping Z-tokens (e.g., z_{42}, z_{87}), suggesting a shared encoding of government-led economic intervention despite lexical variation in the decompressed outputs. 2) **Syntactic alternation:** “*Climate change threatens agricultural productivity and requires immediate attention*” and “*Agricultural productivity is threatened by climate change, which requires immediate action*” map to shared tokens (e.g., z_{156}, z_{203}), indicating that some codes

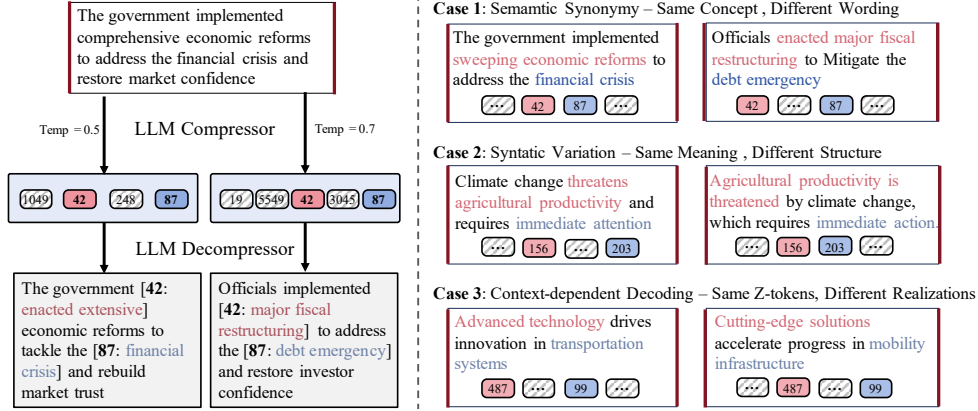


Figure 2: The figure shows the flexibility of Z-tokens. On the **left**, the same sentence can be compressed into different Z-token sequences while preserving meaning after decompression. On the **right**, the same Z-tokens yield different outputs in different contexts. This suggests that Z-tokens are context-dependent rather than chaotic.

may be relatively insensitive to surface word order while preserving higher-level semantic content. 3) **Domain-level variation:** “Advanced technology drives innovation in transportation systems” and “Cutting-edge solutions accelerate progress in mobility infrastructure” align on tokens such as z_{487} , z_{99} , again suggesting reuse at the level of abstract topical meaning rather than exact lexical realization.

Interpretation. These observations suggest that Z-tokens may function as reusable, underspecified semantic codes rather than fixed lexical symbols. In this view, compression abstracts away some surface variability, allowing multiple paraphrastic realizations to share parts of the same discrete representation. We stress that this is an empirical interpretation of the learned code behavior, not a claim that each Z-token carries an invariant standalone meaning.

Measuring contextual regularity. To assess whether this reuse is non-random, we measure the contextual consistency of each Z-token across its corpus occurrences. Let $\mathcal{O}(z_i) = \{o_1, \dots, o_{N_i}\}$ denote the set of occurrences of z_i in a corpus. For each occurrence o_p , let $\text{ctx}(o_p)$ be a fixed-width window of surface text around o_p , and let $\mathcal{F}(\cdot)$ be a frozen semantic encoder (e.g., BERT) that maps text to an embedding. We then define:

$$\mathcal{C}(z_i) = \frac{1}{\binom{N_i}{2}} \sum_{p < q} \text{sim}(\mathcal{F}(\text{ctx}(o_p)), \mathcal{F}(\text{ctx}(o_q))) . \quad (13)$$

A high value of $\mathcal{C}(z_i)$ indicates that occurrences of the same Z-token tend to appear in semantically similar neighborhoods, suggesting structured reuse rather than arbitrary code assignment. On our evaluation set, the mean consistency is 0.75 ± 0.11 , which supports the claim that Z-token reuse exhibits substantial contextual regularity. This context dependence naturally raises the question of how underspecified codes are resolved during decompression. Importantly, Z-token decoding is not performed in isolation. Instead, each output token is conditioned on three sources of information: the complete Z-sequence $Z = (z_1, \dots, z_K)$, the autoregressive text prefix $x_{<t}$, and the base-vocabulary constraint $\mathcal{V}_{\text{base}}$. The full Z-sequence narrows broad semantic possibilities through neighboring codes; the generated prefix progressively commits the decoder to one semantic realization; and restricting logits to the pre-trained vocabulary anchors decoding in well-formed natural language. Together, these constraints convert underspecified latent codes into context-resolved surface outputs.

4 Experiment

4.1 Baseline

Our experiments used the Qwen3-0.6B, 1.7B and 4B. For LoRA, we apply LoRA to all projection linear layers in the LLM and set r to 128, α to 256, and dropout to 0.1. We compared our method with state-of-the-art methods, including ICAE [7], GistToken [10], AutoCompressor [9], and LLOCO [67]: ICAE compresses the input into a fixed-length memory slot; GistToken compresses the input into a fixed-length token using attention masking; AutoCompressor compresses the input in segments into a fixed-length summary vector; and LLOCO uses a fine-tuned AutoCompressor and a base LLM for contextual compression. To ensure a fair comparison, all basic settings were kept consistent across all methods, and the Z-token size is 8k. For experimental environment information, please refer to A.

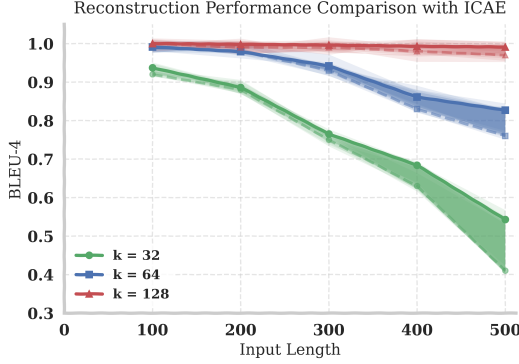


Figure 3: BLEU-4 at different input lengths and compression ratios. The solid line represents our method, and the dashed line represents ICAE.

4.2 Metrics

For text reconstruction and continuation tasks, we use BLEU-4 as the core evaluation metric. Additionally, we report the perplexity score (PPL), which quantifies the difficulty of the model predicting the text. For the QuALITY dataset, we report the exact match (EM) score. For HotpotQA, NarrativeQA, and QASPER, we report the F1 score. For the CNN/DailyMail summarization task, we provide ROUGE-1, ROUGE-2, and ROUGE-L scores.

4.3 Comparison

We first evaluate the model’s reconstruction performance to understand whether the Z-tokens generated by the compressor contain sufficient semantic knowledge to recover the original context. We test on Wikipedia and report the BLEU-4 and CE-Loss results, as shown in Table 1. Our method achieves the highest BLEU-4 score and the lowest CE-Loss at various compression ratios. Notably, the actual compression ratio is even higher due to our use of autoregressive variable-length compression. For example, at a nominal compression ratio of $4\times$, our effective average compression ratio reaches $5.1\times$; and at a compression ratio of $8\times$, it reaches $9.4\times$. These results demonstrate that our variable-length compression method effectively preserves key semantic content while significantly improving compression efficiency.

Figure 3 explores the relationship between BLEU-4 and sequence length. It shows that at a compression ratio of $1/4$, our method is almost unaffected, it can almost perfectly reconstruct the original input. Even at a compression ratio of $16\times$, our method still achieves 54.3, demonstrating the significant performance of our method. Table 2 shows the impact of different LLM sizes on the reconstruction task. It can be seen that the larger the model, the better the reconstruction performance and the lower the perplexity. The simplicity and exclusivity of the reconstruction task may lead to suboptimal generalization. Previous research [7] has shown that text continuation can broadly promote the learning of more general representations because text continuation requires the model to capture not only surface-level correspondences but also the deep discourse and semantic coherence necessary for text continuation. During training, the model observes only the prefix $x_{1:t}$ of a document and learns to predict its remainder $x_{t+1:T}$. We tested this task on Wikipedia, and the results are shown in Table 3. The results show that our model maintains almost the same language modeling ability across different context lengths. Even when the available context is reduced from 1024 to 128, the perplexity only increases by 0.44, indicating that the model can still predict subsequent text coherently and fluently. This demonstrates that the compressed Z-token representation effectively preserves key semantics and contextual dependencies, enabling robust next-text prediction even with limited preceding context.

The summarization task evaluates the model’s ability to abstract high-level content from compressed representations. We conduct experiments on CNN/DailyMail under two paradigms, i.e., direct de-compression and inference de-compression. The results are shown in Table 4. We set the compression ratio to $4\times$, however, due to variable-length compression, the average compression ratio reached

Table 1: Comparison on Wikipedia reconstruction at compression ratios of $4\times$ and $8\times$.

Method	$4\times$		$8\times$	
	BLEU $\uparrow$	CE $\downarrow$	BLEU $\uparrow$	CE $\downarrow$
AutoCompressor [9]	96.44	0.29	92.67	0.38
Gist Token [10]	96.29	0.21	94.63	0.30
ICAIE [7]	98.45	0.04	93.44	0.17
Ours	99.31	0.04	96.28	0.13

Table 2: Performance comparison of reconstruction task performance at different model sizes.

Model	BLEU-4 $\uparrow$	PPL (Ori) $\downarrow$	PPL (Z) $\downarrow$	Gap
Qwen3-0.6B	96.77	30.42	30.97	+0.55
Qwen3-1.7B	97.14	23.75	24.16	+0.41
Qwen3-4B	97.57	16.90	17.14	+0.24

Table 3: Comparison of PPL in the continuation task.

Context	PPL (Ori) $\downarrow$	PPL (Z) $\downarrow$	Gap
1024 $\rightarrow$ 512	29.96	30.12	+0.16
1024 $\rightarrow$ 256	30.42	30.65	+0.23
1024 $\rightarrow$ 128	29.90	30.34	+0.44

6.3 $\times$, while maintaining performance comparable to or even better than the original model. Notably, in the direct decompression, the inference time was reduced from 35 min to 17 min, a speedup of 2 $\times$, demonstrating the efficiency and scalability of our method.

Table 4: Performance comparison of summarization tasks on CNN/DailyMail.

Method	Rouge-1 $\uparrow$	Rouge-2 $\uparrow$	Rouge-L $\uparrow$
Qwen3-0.6B	33.25	16.58	30.83
AutoCompressor [9]	30.28	14.86	28.49
GistToken [10]	29.11	14.57	27.62
ICAE [7]	30.76	15.31	28.84
Ours (w/o infer)	32.58	16.77	29.89
Ours (w/ infer)	<u>32.04</u>	<u>15.57</u>	30.15

Table 5: Long-text QA results under 4 $\times$ compression.

Methods	QuALITY	HotpotQA	NarrativeQA	QASPER
Qwen3-0.6B [68]	38.48	32.18	16.32	18.44
AutoCom [9]	34.43	23.59	14.41	16.47
ICAE [7]	34.68	31.65	15.76	17.62
LLOCO [67]	37.33	33.46	15.88	<u>18.20</u>
Ours (w/o infer)	39.25	<u>33.35</u>	16.24	18.31
Ours (w/ infer)	<u>39.04</u>	33.04	<u>15.93</u>	17.66

For long text question answering, we evaluated our method on the HotpotQA, QuALITY, NarrativeQA, and QASPER datasets. We applied a 4 $\times$ compression ratio to the input and tested it against two paradigms. The results are shown in Table 5. Under the same compression budget, our method outperforms prior compression baselines on QuALITY, NarrativeQA, and QASPER, and remains close to the best baseline on HotpotQA (33.35 vs. 33.46). We also observed that we can achieve a compression ratio of 18 $\times$ for some inputs. This demonstrates that our compressed representation can preserve task-relevant semantics and maintain multi-hop inference capabilities even with significantly reduced text length.

Sliding window We experimented on the QASPER dataset. Due to hardware limitations, we set the window size to 1024 and the stride to 256, enabling the model to process documents incrementally while maintaining semantic continuity. For comparison, we also evaluated a baseline configuration where the entire document was fed into the model at once, as a performance cap for the uncompressed case. The results in Table 6 show that our method uses fewer tokens per inference but achieves comparable performance. This demonstrates that our compression mechanism can be naturally combined with a sliding window strategy to handle inputs exceeding the maximum context window of an LLM.

Table 6: QASPER sliding-window results.

Method	F1 $\uparrow$
Qwen3-0.6B	18.44
ICAE	17.26
Ours	18.25

4.4 Ablation Study.

We first study the effect of scheduled sampling on free-decoding performance in Wikipedia reconstruction. We compare against ICAE, which is trained with teacher forcing and therefore suffers from exposure bias: although decoding is accurate under teacher-forced training, errors accumulate rapidly during free decoding. To improve robustness, we adopt scheduled sampling in training (Section 3.2), gradually increasing the sampling probability from 0 to 0.5. We do not push p to 1, since more aggressive sampling can further stabilize free decoding but often slows convergence and degrades fluency. Under this setting, Table 7 shows that scheduled sampling substantially improves free-decoding reconstruction, making the model more robust to autoregressive error accumulation. We further study the effect of Z-token vocabulary size on HotpotQA using settings of 1k, 2k, 4k, 8k, and 10k, as reported in Table 8. When the Z-token set is too small, the compressed representation becomes overly restrictive and fails to preserve enough relational and contextual information, leading to worse QA performance. As the vocabulary size increases, performance improves accordingly, but the gain becomes marginal beyond 8k, suggesting that the representation has already captured the key reasoning information. This result indicates a practical trade-off between compression capacity and computational cost. Given the highly structured nature of Wikipedia data, we also tested it on the less structured HumanEval; the results are shown in Appendix B, demonstrating the robustness of our method.

Practical Efficiency Analysis A natural question is whether context compression remains useful as model size increases. In larger models, a greater fraction of runtime may shift from attention

Table 7: Free-decoding BLEU-4 on Wikipedia under different input lengths.

Method	100	200	300	400	500
ICAE [7]	29.46	21.88	14.53	11.27	8.96
Ours	58.24	52.23	46.66	41.84	37.66

Table 8: Effect of different Z-token counts on HotpotQA.

Dataset	1k	2k	4k	8k	10k
HotpotQA (F1)	16.23	22.45	27.96	33.35	33.27

Table 9: Results under different KV-cache budgets / compression ratios. A: Acc, M: memory (GB), T: inference time (s).

Data	Method	25%	50%	Base
		A/M/T	A/M/T	A/M/T
PIQA	Ours	62.70/10.63/142.57	66.53/14.28/174.32	67.30/27.74/198.26
	H2O	61.48/11.49/134.28	65.32/16.91/170.21	
RTE	Ours	68.46/9.30/56.88	71.70/14.20/70.28	73.64/25.41/82.12
	H2O	68.01/10.83/58.46	71.35/15.16/66.88	

to MLP computation, which could potentially weaken the benefit of shortening the input context. To examine this issue, we report the Attention/MLP time ratio for Qwen3-4B and Qwen3-8B at different input lengths in Table 10. The results show that, although MLP computation becomes increasingly important in larger models, attention still occupies a substantial fraction of runtime and grows consistently with sequence length. In particular, as the input length increases from 8×256 to 8×1024 , the attention ratio rises markedly for both Qwen3-4B and Qwen3-8B. This indicates that shortening the effective context remains beneficial even at larger model scales, and therefore the proposed compression mechanism retains its practical value beyond small or medium-sized LLMs.

Inference efficiency analysis To evaluate the practical efficiency of compressed representations, we compare standard long-context generation with generation from compressed Z-token sequences on Qwen3-4B under the same output length of 64 tokens. For our method, the reported time includes both the autoregressive compression step and the decompression/generation step, while throughput and memory are measured during the generation stage. As shown in Table 11, our method consistently reduces end-to-end latency and generation-stage peak GPU memory across all input lengths, while improving generation throughput. The advantage becomes more pronounced as the context grows. At 30k tokens, latency decreases from 22.67 s to 17.21 s, throughput increases from 24.09 to 47.69 tokens/s, and peak memory decreases from 56.48 GB to 25.40 GB. These results suggest that Z-token compression remains beneficial even when compression overhead is included, especially for long-context and memory-constrained inference.

Adaptive Z-token allocation. To examine whether variable-length compression reflects information density rather than nearly uniform allocation, we conduct a controlled analysis under the same nominal $4 \times$ compression setting as the main experiments. Using 10,000 fixed-length segments from Wikipedia and HotpotQA, we analyze the relationship between allocated Z-token count and two segment-level proxies of semantic density: surprisal and entity-rare density. Since all segments have the same length, the observed trends cannot be attributed to trivial length effects. As shown in Table 12, the allocated Z-token count is positively correlated with both proxies across both domains. Moreover, the trend across surprisal buckets is monotonic: segments with higher surprisal consistently receive more latent tokens. These results suggest that the learned compressor does not allocate latent budget uniformly, but instead assigns more Z-tokens to segments that are harder to predict or contain denser factual content. This provides empirical support for the intended allocation principle behind our variable-length design. Full details and additional results are given in Appendix C.

Adaptive Z-token allocation. To examine whether variable-length compression reflects information density rather than nearly uniform allocation, we conduct a controlled analysis under the same nominal $4 \times$ compression setting as the main experiments. Using 10,000 fixed-length segments from Wikipedia and HotpotQA, we analyze the relationship between allocated Z-token count and two segment-level proxies of semantic density: surprisal and entity-rare density. Since all segments have the same length, the observed trends cannot be attributed to trivial length effects. As shown in Table 12, the allocated Z-token count is positively correlated with both proxies across both domains. Moreover, the trend across surprisal buckets is monotonic: segments with higher surprisal consistently receive more latent tokens. These results suggest that the learned compressor does not allocate latent budget uniformly, but instead assigns more Z-tokens to segments that are harder to predict or contain denser factual content. This provides empirical support for the intended allocation principle behind our variable-length design. Full details and additional results are given in Appendix C.

Adaptive Z-token allocation. To examine whether variable-length compression reflects information density rather than nearly uniform allocation, we conduct a controlled analysis under the same nominal $4 \times$ compression setting as the main experiments. Using 10,000 fixed-length segments from Wikipedia and HotpotQA, we analyze the relationship between allocated Z-token count and two segment-level proxies of semantic density: surprisal and entity-rare density. Since all segments have the same length, the observed trends cannot be attributed to trivial length effects. As shown in Table 12, the allocated Z-token count is positively correlated with both proxies across both domains. Moreover, the trend across surprisal buckets is monotonic: segments with higher surprisal consistently receive more latent tokens. These results suggest that the learned compressor does not allocate latent budget uniformly, but instead assigns more Z-tokens to segments that are harder to predict or contain denser factual content. This provides empirical support for the intended allocation principle behind our variable-length design. Full details and additional results are given in Appendix C.

4.5 Conclusion.

This paper proposes a discretization method that integrates compression and generation into one LLM, enabling the model to naturally compress long texts into variable-length, controllable Z-tokens before downstream tasks. Extensive experiments on multiple tasks demonstrate that the method significantly reduces the latency and memory cost of downstream long-context inference while maintaining good task performance. These results show that allowing the LLM to infer its own compact latent language provides a promising direction for efficient long-context modeling. Future work could consider developing a global coordination mechanism across windows and a task-aware compression strategy to better preserve the structure required for specific downstream objectives.

Table 10: Attention/MLP time ratio at different input lengths for Qwen3-4B and 8B. Higher attention ratios at longer inputs indicate that reducing effective context length remains beneficial even for larger models.

Model	Input	Attn / MLP (%)
Qwen3-8B	$8 \times 256/512/1024$	43.8/47.9/54.4
Qwen3-4B	$8 \times 256/512/1024$	47.7/52.9/60.4

Table 11: On Qwen3-4B, we compare long-context generation with and without compression, reporting inference time, throughput, and peak GPU memory.

Input Length	Baseline			Ours (Compressed Context)		
	Time (s)	Tok/s	Mem. (GB)	Time (s)	Tok/s	Mem. (GB)
10k	11.36	45.09	24.86	9.25	70.61	12.77
20k	16.11	34.06	41.82	12.39	56.28	19.17
30k	22.67	24.09	56.48	17.21	47.69	25.40

Table 12: Correlation between allocated Z-token count and semantic-density proxies.

Dataset	Proxy	Pearson r	Spearman ρ
Wikipedia	Surprisal	0.54	0.57
HotpotQA	Surprisal	0.49	0.52
Wikipedia	Entity-rare density	0.36	0.39
HotpotQA	Entity-rare density	0.43	0.46

References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [3] Zikai Song, Junqing Yu, Yi-Ping Phoebe Chen, and Wei Yang. Transformer tracking with cyclic shifting window attention. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8791–8800, 2022.
- [4] Zikai Song, Run Luo, Junqing Yu, Yi-Ping Phoebe Chen, and Wei Yang. Compact transformer tracker with correlative masked modeling. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 2321–2329, 2023.
- [5] Sicheng Feng, Kaiwen Tuo, Song Wang, Lingdong Kong, Jianke Zhu, and Huan Wang. Rewardmap: Tackling sparse rewards in fine-grained visual reasoning via multi-stage reinforcement learning. *arXiv preprint arXiv:2510.02240*, 2025.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [7] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model, 2024.
- [8] Charlie Snell, Dan Klein, and Ruiqi Zhong. Learning by distilling context, 2022.
- [9] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts, 2023.
- [10] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens, 2024.
- [11] Sicheng Feng, Song Wang, Shuyi Ouyang, Lingdong Kong, Zikai Song, Jianke Zhu, Huan Wang, and Xinchao Wang. Can mllms guide me home? a benchmark study on fine-grained visual reasoning from transit maps. *arXiv preprint arXiv:2505.18675*, 2025.
- [12] Yangliu Hu, Zikai Song, Na Feng, Yawei Luo, Junqing Yu, Yi-Ping Phoebe Chen, and Wei Yang. Sf2t: Self-supervised fragment finetuning of video-llms for fine-grained understanding. *arXiv preprint arXiv:2504.07745*, 2025.
- [13] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster, 2023.
- [14] Yiwu Zhong, Zhuoming Liu, Yin Li, and Liwei Wang. Aim: Adaptive inference of multi-modal llms via token merging and pruning, 2025.
- [15] Sehoon Kim, Sheng Shen, David Thorsley, Amir Gholami, Woosuk Kwon, Joseph Hassoun, and Kurt Keutzer. Learned token pruning for transformers, 2022.
- [16] Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. Compressing context to enhance inference efficiency of large language models, 2023.
- [17] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models, 2022.

- [18] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, et al. On the opportunities and risks of foundation models, 2022.
- [19] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression, 2024.
- [20] Zikai Song, Ying Tang, Run Luo, Lintao Ma, Junqing Yu, Yi-Ping Phoebe Chen, and Wei Yang. Autogenic language embedding for coherent point tracking. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 2021–2030, 2024.
- [21] Yunyao Zhang, Zikai Song, Hang Zhou, Wenfeng Ren, Yi-Ping Phoebe Chen, Junqing Yu, and Wei Yang. $ga - s^3$: Comprehensive social network simulation with group agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 8950–8970, 2025.
- [22] Zikai Song, Run Luo, Lintao Ma, Ying Tang, Yi-Ping Phoebe Chen, Junqing Yu, and Wei Yang. Temporal coherent object flow for multi-object tracking. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 6978–6986, 2025.
- [23] Yunyao Zhang, Xinglang Zhang, Junxi Sheng, Wenbing Li, Junqing Yu, Yi-Ping Phoebe Chen, Wei Yang, and Zikai Song. Semantic-aware logical reasoning via a semiotic framework, 2026.
- [24] Xin Liu, Jie Liu, Jie Tang, and Gangshan Wu. Catanet: Efficient content-aware token aggregation for lightweight image super-resolution, 2025.
- [25] Shigui Li, Wei Chen, and Delu Zeng. Evodiff: Entropy-aware variance optimized diffusion inference. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [26] Shigui Li, Linzhang Lu, Xing Qiu, Zhen Chen, and Delu Zeng. Tighter bound estimation for efficient biquadratic optimization over unit spheres. *Journal of Global Optimization*, 90(2):323–353, 2024.
- [27] Liliang Ye, Yunyao Zhang, Yafeng Wu, Yi-Ping Phoebe Chen, Junqing Yu, Wei Yang, and Zikai Song. Mvp: Winning solution to smp challenge 2025 video track. *arXiv preprint arXiv:2507.00950*, 2025.
- [28] Yunyao Zhang, Yihao Ai, Zuocheng Ying, Qirui Mi, Junqing Yu, Wei Yang, and Zikai Song. Coupling macro dynamics and micro states for long-horizon social simulation, 2026.
- [29] Yongming Rao, Wenliang Zhao, Benlin Liu, Jiwen Lu, Jie Zhou, and Cho-Jui Hsieh. Dynamicvit: Efficient vision transformers with dynamic token sparsification, 2021.
- [30] Eugene Kwek and Wenpeng Yin. Compact: Common-token optimized model pruning across channels and tokens, 2025.
- [31] Woomin Song, Seunghyuk Oh, Sangwoo Mo, Jaehyung Kim, Sukmin Yun, Jung-Woo Ha, and Jinwoo Shin. Hierarchical context merging: Better long context understanding for pre-trained llms, 2024.
- [32] Xinglang Zhang, Yunyao Zhang, ZeLiang Chen, Junqing Yu, Wei Yang, and Zikai Song. Logical phase transitions: Understanding collapse in llm logical reasoning, 2026.
- [33] Zikai Song, Junqing Yu, Yi-Ping Phoebe Chen, Wei Yang, and Xinchao Wang. Hypergraph-state collaborative reasoning for multi-object tracking, 2026.
- [34] Zixu Li, Yupeng Hu, Zhiwei Chen, Qinlei Huang, Guozhi Qiu, Zhiheng Fu, and Meng Liu. Retrack: Evidence-driven dual-stream directional anchor calibration network for composed video retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 23373–23381, 2026.
- [35] Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. Long context compression with activation beacon, 2024.

- [36] Shigui Li, Wei Chen, and Delu Zeng. Scire-solver: Accelerating diffusion models sampling by score-integrand solver with recursive difference. *arXiv preprint arXiv:2308.07896*, 2023.
- [37] Zong Ke, Shicheng Zhou, Yining Zhou, Chia Hong Chang, and Rong Zhang. Detection of ai deepfake and fraud in online payments using gan-based models. In *2025 8th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*, pages 1786–1790. IEEE, 2025.
- [38] Zixu Li, Yupeng Hu, Zhiwei Chen, Shiqi Zhang, Qinlei Huang, Zhiheng Fu, and Yinwei Wei. Habit: Chrono-synergia robust progressive learning framework for composed image retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 6762–6770, 2026.
- [39] Yunyao Zhang, Zuocheng Ying, Xinglang Zhang, Junqing Yu, Peng Fang, Xu Chen, Wei Yang, and Zikai Song. Intervensim: Intervention-aware social network simulation for opinion dynamics, 2026.
- [40] Zongqian Li, Yixuan Su, and Nigel Collier. 500xcompressor: Generalized prompt compression for large language models, 2024.
- [41] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression, 2024.
- [42] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling, 2019.
- [43] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer, 2020.
- [44] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers, 2020.
- [45] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019.
- [46] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020.
- [47] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences, 2021.
- [48] Wenbing Li, Hang Zhou, Junqing Yu, Zikai Song, and Wei Yang. Coupled mamba: Enhanced multimodal fusion with coupled state space model. *Advances in Neural Information Processing Systems*, 37:59808–59832, 2024.
- [49] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025.
- [50] Gongfan Fang, Hongxu Yin, Saurav Muralidharan, Greg Heinrich, Jeff Pool, Jan Kautz, Pavlo Molchanov, and Xinchao Wang. Maskllm: Learnable semi-structured sparsity for large language models, 2024.
- [51] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.
- [52] Wenbing Li, Zikai Song, Hang Zhou, Yunyao Zhang, Junqing Yu, and Wei Yang. Lora-mixer: Coordinate modular lora experts through serial attention routing, 2025.
- [53] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models, 2024.

- [54] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning, 2018.
- [55] Łukasz Kaiser and Samy Bengio. Discrete autoencoders for sequence models, 2018.
- [56] Łukasz Kaiser, Aurko Roy, Ashish Vaswani, Niki Parmar, Samy Bengio, Jakob Uszkoreit, and Noam Shazeer. Fast decoding in sequence models using discrete latent variables, 2018.
- [57] Qijiong Liu, Hengchang Hu, Jiahao Wu, Jieming Zhu, Min-Yen Kan, and Xiao-Ming Wu. Discrete semantic tokenization for deep ctr prediction, 2024.
- [58] Tiancheng Zhao, Kyusong Lee, and Maxine Eskenazi. Unsupervised discrete sentence representation learning for interpretable neural dialog generation, 2018.
- [59] Yupeng Hu, Zixu Li, Zhiwei Chen, Qinlei Huang, Zhiheng Fu, Mingzhu Xu, and Liqiang Nie. Refine: Composed video retrieval via shared and differential semantics enhancement. *ACM Transactions on Multimedia Computing, Communications and Applications*, 2026.
- [60] Alexander H. Liu, SouYoung Jin, Cheng-I Jeff Lai, Andrew Rouditchenko, Aude Oliva, and James Glass. Cross-modal discrete representation learning, 2021.
- [61] Zhiwei Chen, Yupeng Hu, Zixu Li, Zhiheng Fu, Xuemeng Song, and Liqiang Nie. Offset: Segmentation-based focus shift revision for composed image retrieval. In *Proceedings of the ACM International Conference on Multimedia*, page 6113–6122, 2025.
- [62] Zhiwei Chen, Yupeng Hu, Zhiheng Fu, Zixu Li, Jiale Huang, Qinlei Huang, and Yinwei Wei. Intent: Invariance and discrimination-aware noise mitigation for robust composed image retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 20463–20471, 2026.
- [63] Shuning Jin, Sam Wiseman, Karl Stratos, and Karen Livescu. Discrete latent variable representations for low-resource text classification, 2020.
- [64] Guozhi Qiu, Zhiwei Chen, Zixu Li, Qinlei Huang, Zhiheng Fu, Xuemeng Song, and Yupeng Hu. Melt: Improve composed image retrieval via the modification frequentation-rarity balance network. *arXiv preprint arXiv:2603.29291*, 2026.
- [65] Qianyun Yang, Zhiwei Chen, Yupeng Hu, Zixu Li, Zhiheng Fu, and Liqiang Nie. Stable: Efficient hybrid nearest neighbor search via magnitude-uniformity and cardinality-robustness. *arXiv preprint arXiv:2604.01617*, 2026.
- [66] Mingyu Zhang, Zixu Li, Zhiwei Chen, Zhiheng Fu, Xiaowei Zhu, Jiajia Nie, Yinwei Wei, and Yupeng Hu. Hint: Composed image retrieval with dual-path compositional contextualized network. *arXiv preprint arXiv:2603.26341*, 2026.
- [67] Sijun Tan, Xiuyu Li, Shishir Patil, Ziyang Wu, Tianjun Zhang, Kurt Keutzer, Joseph E. Gonzalez, and Raluca Ada Popa. Lloco: Learning long contexts offline, 2024.
- [68] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.

A Implementation details.

All experiments used six NVIDIA GPUs: two A800-80G and four V100-32G. Each configuration is equipped with one Intel Xeon Gold 6348 CPU. The software environment includes Ubuntu 22.04 LTS, Python 3.12, PyTorch 2.5.1, and CUDA 12.4. Unless otherwise specified, the base LLM used in the experiments is Qwen3-0.6B.

During the reconstruction and continuation of pre-training, we used Qwen3-0.6B as the base model and trained the compressor-decompressor framework using LoRA. Training was performed on two NVIDIA A800 GPUs with a precision of bfloat16, and FlashAttention and gradient checkpointing were enabled to improve memory efficiency. The batch size per GPU was set to 4, resulting in a global batch size of 8. The input length of the pre-training snippets was fixed at 1024 tokens. The overall training time was approximately 3-4 days. While this training scale is significantly larger than standard downstream fine-tuning, it is appropriate for our setup because the model needs to adapt not only to supervised tasks but also to achieve stable compression-decompression behavior on discrete, variable-length latent sequences.

B Generalization

We further evaluate the generalization ability of our method on HumanEval, which contains relatively unstructured code-generation prompts and is therefore substantially different from the long-form text corpora used in pretraining. As shown in Table 13, under $4\times$ compression, AutoCompressor leads to a noticeable drop in pass@1 accuracy, whereas our method preserves performance more effectively and remains substantially closer to the base model. These results suggest that the learned compression mechanism generalizes beyond standard document reconstruction and can retain useful information even in less structured input domains.

Table 13: HumanEval results, reported with pass@1.

Dataset	Base Model	AutoCompressor	Ours
HumanEval	18.90	12.64	16.85

C Analyzing Variable-Length Z-Token Allocation

A central question behind variable-length compression is whether the compressor allocates more Z-tokens to semantically denser segments, rather than producing lengths that are nearly uniform or driven by superficial statistics. To probe this behavior, we conduct a controlled diagnostic study calibrated to the main experimental regime, namely nominal $4\times$ compression. We emphasize that this analysis is intended as a targeted diagnostic of allocation behavior under the same compression regime as the main experiments, rather than as a definitive annotation-based measure of semantic density.

We construct 10,000 diagnostic segments, including 5000 Wikipedia-style segments from reconstruction scenarios and 5000 HotpotQA-style segments from long-context question answering scenarios. Each segment contains exactly 256 input tokens. For each segment i , we record the allocated Z-token count k_i together with two segment-level proxies of semantic density:

- **Segment surprisal**: the mean token-level negative log-likelihood under the base model;
- **Entity-rare density**: the fraction of named entities and rare words in the segment.

We then report Pearson and Spearman correlations between each proxy and k_i , and additionally summarize the mean allocated Z-token count across surprisal quintiles. Because all segments have the same input length, the observed trends cannot be explained by trivial dependence on raw segment length. We adopt this controlled protocol to isolate allocation behavior under matched segment length and compression budget, which would otherwise be more difficult to disentangle from confounding factors in full end-to-end task traces.

Across both domains, the allocated Z-token count is positively associated with multiple semantic-density proxies, and the surprisal-bucket trend is monotonic. These results provide supporting evidence that Z-token allocation is correlated with segment-level information density, rather than being approximately uniform across segments. At the same time, we do not claim that these

Table 14: Correlation between allocated Z-token count and semantic-density proxies in the controlled diagnostic study.

Dataset	Proxy	Pearson r	Spearman ρ
Wikipedia	Segment surprisal	0.54	0.57
HotpotQA	Segment surprisal	0.49	0.52
Wikipedia	Entity-rare density	0.36	0.39
HotpotQA	Entity-rare density	0.43	0.46

Surprisal quintile	Mean allocated Z-tokens	Trend bar
Q1 (lowest)	37.1	=====
Q2	41.0	=====
Q3	46.4	=====
Q4	52.2	=====
Q5 (highest)	59.8	=====

Figure 4: Binned trend plot showing the relationship between surprisal quintile and allocated Z-token count. Higher-surprisal segments consistently receive more Z-tokens.

proxies fully capture semantic complexity, nor that this controlled analysis replaces a fully annotated semantic-density evaluation. Instead, the results should be understood as evidence that the learned variable-length compressor behaves consistently with the intended allocation principle: segments that are harder to predict or contain denser factual content tend to receive a larger share of the latent Z-token budget.

D Variable-length compression.

Although our compressor is implemented autoregressively, its behavior can be understood from a simple constrained information allocation perspective. The key question is not merely how to compress an input sequence $X = \{x_1, \dots, x_N\}$ into a shorter latent sequence $Z = \{z_1, \dots, z_K\}$, but how to allocate a limited compression budget across parts of the input so that semantic distortion is minimized.

Consider partitioning X into M local segments,

$$X = \{X^{(1)}, X^{(2)}, \dots, X^{(M)}\},$$

and suppose the compressor allocates K_i latent tokens to segment $X^{(i)}$, with

$$\sum_{i=1}^M K_i = K.$$

Let $D_i(K_i)$ denote the reconstruction or task-induced distortion contributed by segment $X^{(i)}$ when it is represented by K_i Z-tokens. A natural objective is then

$$\min_{\{K_i\}} \sum_{i=1}^M D_i(K_i) \quad \text{s.t.} \quad \sum_{i=1}^M K_i \leq \frac{N}{r},$$

where r is the target compression ratio. This is a standard budgeted allocation problem: the model must decide where the limited latent capacity should be spent.

To make this objective interpretable, we introduce a segment-wise semantic complexity measure

$$H_i \triangleq H(X^{(i)} | X^{(<i)}),$$

which can be viewed as the conditional uncertainty or information density of segment $X^{(i)}$ given its preceding context. Intuitively, segments with larger H_i are harder to predict from context and therefore require more latent capacity to preserve their semantics. We assume that the distortion for each segment decreases with the number of allocated Z-tokens and that the reduction is larger for semantically denser segments. A simple form satisfying this intuition is

$$D_i(K_i) = H_i \exp(-\beta K_i),$$

where $\beta > 0$ controls how efficiently each additional Z-token reduces distortion.

Under this model, the constrained problem becomes

$$\min_{\{K_i\}} \sum_{i=1}^M H_i \exp(-\beta K_i) \quad \text{s.t.} \quad \sum_{i=1}^M K_i \leq \frac{N}{r}.$$

Its Lagrangian is

$$\mathcal{L}(\{K_i\}, \lambda) = \sum_{i=1}^M H_i \exp(-\beta K_i) + \lambda \left(\sum_{i=1}^M K_i - \frac{N}{r} \right),$$

with $\lambda \geq 0$. Taking derivatives with respect to K_i gives

$$\frac{\partial \mathcal{L}}{\partial K_i} = -\beta H_i \exp(-\beta K_i) + \lambda.$$

At the optimum, for every active segment,

$$\beta H_i \exp(-\beta K_i^*) = \lambda,$$

which yields

$$K_i^* = \frac{1}{\beta} (\log(\beta H_i) - \log \lambda).$$

Therefore,

$$K_i^* \text{ is monotonically increasing in } H_i.$$

This directly implies that, under a fixed global compression budget, semantically denser segments should receive more Z-tokens, while more predictable or redundant segments should receive fewer.

This view provides a theoretical interpretation of our design. The autoregressive compressor with the special token [EOS-Z] determines the realized length K adaptively, while the length regularizer

$$L_{\text{len}} = \left(\frac{K}{N} - \frac{1}{r} \right)^2$$

acts as a soft relaxation of the hard budget constraint $\sum_i K_i \leq N/r$. In other words, our training objective can be viewed as optimizing a soft rate–distortion trade-off: the model is encouraged to minimize downstream reconstruction or task loss while spending its latent Z-token budget where it reduces semantic distortion the most.

Importantly, this analysis does not claim that each Z-token corresponds to a fixed symbolic meaning, nor that the trained transformer exactly solves the above optimization. Rather, it explains why variable-length compression is the natural outcome of an adaptive latent bottleneck: if different parts of the input carry different amounts of conditional information, then an optimal compressor should not allocate the same number of latent tokens everywhere. Our method implements this principle in a discrete autoregressive form that is compatible with the native generation process of LLMs.

E Dataset statistics.

Table 15 summarizes the main downstream long-context datasets used in our experiments. These benchmarks cover a variety of tasks, including abstract text summarization (CNN/DailyMail), multi-hop question answering (HotpotQA), long narrative comprehension (NarrativeQA), scientific document question answering (QASPER), and the more challenging long-context multiple-choice reading comprehension (QUALITY). The reported dataset sizes follow standard publicly available partitions.

During the reconstruction and continuation pre-training phases, we used a large subset of Wikipedia containing approximately 300,000 full-text articles randomly selected from various domains. To construct long-context training instances, each article was divided into blocks of 1024 units in length, with 256 overlapping terms between adjacent blocks within the same document to maintain local semantic continuity. In practice, each article generated an average of about 3 to 6 training segments, totaling approximately 1.45 million segments.

Table 15: Statistics of the downstream long-context datasets used in our experiments. The reported sizes follow standard public splits; exact numbers may vary slightly depending on preprocessing.

Dataset	Task	Train	Valid	Test	Output
CNN/DailyMail	Summarization	287,227	13,368	11,490	Summary
HotpotQA	Multi-hop QA	90,447	7,405	7,405	Answer
NarrativeQA	Long-form QA	32,747	3,461	10,557	Answer
QASPER	Scientific QA	2,593	1,006	1,451	Answer
QuALITY	Multiple-choice QA	265	265	265	Option

F Training and Inference Pipeline

Algorithm 1 Training the Z-token compressor, decompressor, and inferencer

Require: Pretrained LLM backbone f_Θ ; base vocabulary V_{base} ; Z-token vocabulary V_Z ; training examples (X, Y) ; target compression ratio r .

- 1: Initialize three LoRA adapter sets: compressor adapter Δ_ϕ , decompressor adapter Δ_θ , and inferencer adapter Δ_ψ .
- 2: Extend the output vocabulary with V_Z for compressor and inferencer generation.
- 3: **for** each minibatch **do**
- 4: **Compression:** activate Δ_ϕ and generate a variable-length Z-token sequence

$$Z = (z_1, \dots, z_K), \quad z_t \in V_Z,$$

by autoregressive decoding from $p_\phi(Z | X)$ until [EOS-Z] is emitted.

- 5: Use Gumbel-Softmax with a straight-through estimator to obtain differentiable Z-token embeddings during training.
- 6: Compute the length regularization term

$$\mathcal{L}_{\text{len}} = \left(\frac{K}{|X|} - \frac{1}{r} \right)^2.$$

- 7: **Decompression / reconstruction:** activate Δ_θ and decode from Z to the target natural-language sequence.
- 8: **if** reconstruction or continuation training **then**
- 9: Set the target sequence to the original or continued text and compute

$$\mathcal{L}_{\text{tr}} = - \sum_t \log p_\theta(y_t | y_{<t}, Z).$$

- 10: **else if** downstream supervised training **then**
- 11: Set the target sequence to the task output Y and compute the same conditional generation loss.
- 12: **end if**
- 13: Add codebook usage and commitment regularizers $\mathcal{L}_{\text{KL}}$ and $\mathcal{L}_{\text{com}}$.
- 14: Update Δ_ϕ and Δ_θ using

$$\mathcal{L}_{\text{comp-dec}} = \mathcal{L}_{\text{tr}} + \lambda \mathcal{L}_{\text{KL}} + \beta \mathcal{L}_{\text{com}} + \gamma \mathcal{L}_{\text{len}}.$$

- 15: **Optional Z-space inference training:** compress both the prompt/input X and the target response Y into Z^p and Z^r using Δ_ϕ .
- 16: Activate Δ_ψ and train the inferencer to predict the response codes:

$$\mathcal{L}_{\text{infer}} = - \sum_t \log p_\psi(z_t^r | Z^p, z_{<t}^r).$$

- 17: **end for**
-

Algorithm 2 Inference with adapter switching

Require: Input context or prompt X ; pretrained LLM backbone f_Θ ; LoRA adapters $\Delta_\phi, \Delta_\theta, \Delta_\psi$.

- 1: **Step 1: Compress.** Activate compressor adapter Δ_ϕ and generate

$$Z^p \sim p_\phi(Z \mid X), \quad Z^p \subset V_Z.$$

- 2: **Mode A: Direct decompression / direct task inference.**

- 3: Activate decompressor adapter Δ_θ and generate the output in the base vocabulary:

$$\hat{Y} \sim p_\theta(Y \mid Z^p), \quad \hat{Y} \subset V_{\text{base}}.$$

- 4: **Mode B: Z-space autoregressive inference.**

- 5: Activate inferencer adapter Δ_ψ and generate response Z-tokens:

$$\hat{Z}^r \sim p_\psi(Z^r \mid Z^p), \quad \hat{Z}^r \subset V_Z.$$

- 6: Activate decompressor adapter Δ_θ and decode the response codes into natural language:

$$\hat{Y} \sim p_\theta(Y \mid \hat{Z}^r), \quad \hat{Y} \subset V_{\text{base}}.$$

- 7: **return** $\hat{Y}$.
-

All three functional roles are implemented by the same pretrained LLM backbone with different LoRA adapters. The compressor, decompressor, and Z-space inferencer have their own adapter parameters, denoted by Δ_ϕ , Δ_θ , and Δ_ψ , respectively. At inference time, the backbone parameters are shared and frozen, and the model switches between these LoRA adapters depending on the current stage. This design avoids maintaining three independent full LLMs while still allowing each role to specialize in a different mapping: natural language to Z-tokens, Z-tokens to natural language, and Z-tokens to Z-tokens.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction clearly present the main contribution of the paper: adapting an off-the-shelf LLM into a token compressor and decompressor via variable-length discrete latent Z-tokens, and the empirical sections support these claims across reconstruction and downstream tasks.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The paper discusses practical scope and remaining challenges, including hardware constraints, sliding-window settings, model scale effects, and future directions such as global coordination across windows and task-aware compression, which together clarify current limitations and scope.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The paper states the assumptions underlying the theoretical motivation for variable-length compression in the main text and provides further derivation and discussion in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper describes the model setup, training framework, datasets, baselines, compression settings, LoRA configuration, evaluation metrics, and implementation environment in sufficient detail to understand and reproduce the main experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

- (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We will release the code and provide sufficient instructions, including the environment setup, data preparation, and commands needed to reproduce the main experimental results.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper reports the training and evaluation settings needed to understand the experiments, including model sizes, LoRA configuration, compression ratios, window sizes, metrics, hardware setup, and core implementation choices, with additional details provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper states that experiments were independently repeated three times and that the reported scores are averaged across runs, providing information about result stability.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The appendix specifies the compute environment, including GPU types, CPU, operating system, software stack, training precision, and approximate training time.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: To the best of our knowledge, the work conforms to the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper is primarily a methodological contribution focused on efficient long-context modeling and does not include a dedicated broader impact discussion.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a high-risk dataset or model requiring special safeguards as part of the submission.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper properly cites the pretrained models, datasets, and prior methods used in experiments, and uses standard research assets in a conventional academic setting.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not present a new released dataset or benchmark asset as a primary contribution.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The paper explicitly describes the use of pretrained LLMs as the core methodological component, including their roles as compressor, decompressor, and inferencer modules.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.